tf.compat.v1.variable_creator_scope Stay organized with collections Save and categorize content based on your preferences.

https://www.tensorflow.org/api_docs/python/tf/compat/v1/variable_creator_scope

> *Scope which defines a variable creation function to be used by variable().*

Function Signatures

```
tf.compat.v1.variable_creator_scope( variable_creator )
def variable_creator(next_creator, **kwargs)
```

Code Examples

```
@tf_contextlib.contextmanager
tf.compat.v1.variable_creator_scope(
variable_creator
)
```

```
@tf_contextlib.contextmanager
```

```
tf.compat.v1.variable_creator_scope(
variable_creator
)
```

```
def variable_creator(next_creator, **kwargs)
```

```
def variable_creator(next_creator, **kwargs)
```

Documentation

Scope which defines a variable creation function to be used by variable().

variable_creator is expected to be a function with the following signature:

The creator is supposed to eventually call the next_creator to create a variable if it does want to create a variable and not call Variable or ResourceVariable directly. This helps make creators composable. A creator may choose to create multiple variables, return already existing variables, or simply register that a variable was created and defer to the next creators in

line. Creators can also modify the keyword arguments seen by the next creators.

Custom getters in the variable scope will eventually resolve down to these custom creators when they do create variables.

The valid keyword arguments in kwds are:

- initial_value: A Tensor, or Python object convertible to a Tensor,

which is the initial value for the Variable. The initial value must have a shape specified unless validate_shape is set to False. Can also be a callable with no argument that returns the initial value when called. In that case, dtype must be specified. (Note that initializer functions from init_ops.py must first be bound to a shape before being used here.)

- trainable: If True, the default, also adds the variable to the graph

collection GraphKeys.TRAINABLE_VARIABLES. This collection is used as the default list of variables to use by the Optimizer classes. trainable defaults to True, unless synchronization is set to ON_READ, in which case it defaults to False.

- collections: List of graph collections keys. The new variable is added to

these collections. Defaults to [GraphKeys.GLOBAL_VARIABLES].

- validate_shape: If False, allows the variable to be initialized with a

value of unknown shape. If True, the default, the shape of initial_value must be known.

- caching_device: Optional device string describing where the Variable

should be cached for reading. Defaults to the Variable's device.
If not None, caches on another device. Typical use is to cache
on the device where the Ops using the Variable reside, to deduplicate
copying through Switch and other conditional statements.

- name: Optional name for the variable. Defaults to 'Variable' and gets

uniquified automatically.

- dtype: If set, initial_value will be converted to the given type.

If None, either the datatype will be kept (if initial_value is
a Tensor), or convert_to_tensor will decide.

- constraint: A constraint function to be applied to the variable after

updates by some algorithms.

- use_resource: if True, a ResourceVariable is always created.
- synchronization: Indicates when a distributed a variable will be

aggregated. Accepted values are constants defined in the class
tf.VariableSynchronization. By default the synchronization is set to
AUTO and the current DistributionStrategy chooses
when to synchronize.

- aggregation: Indicates how a distributed variable will be aggregated.

Accepted values are constants defined in the class
tf.VariableAggregation.

This set may grow over time, so it's important the signature of creators is as mentioned above.

Yields